

Introduction to Tries

1 What Is a Trie?

Trie

A **trie** is a tree-based data structure used to store a collection of strings. Each edge represents a character, and each root-to-node path represents a prefix.

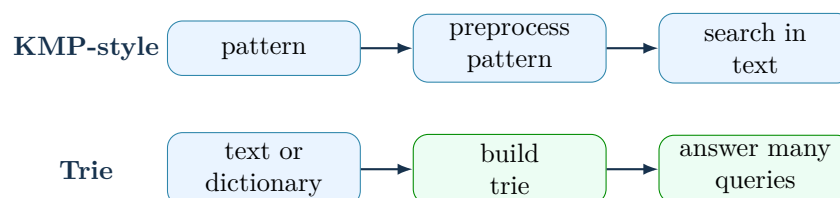
The name is associated with **retrieval**: a trie makes it efficient to retrieve words, prefixes, and dictionary entries.

A trie is useful when:

- the dictionary or text collection is fixed;
- many search queries must be answered;
- prefix queries are important;
- many words share common prefixes.

2 Trie vs Pattern-Preprocessing

String matching algorithms such as KMP preprocess a pattern, then search inside a text. A trie takes the complementary viewpoint: preprocess the stored collection, then answer many query strings.



When a Trie Fits

Use a trie when the stored collection changes slowly but prefix or word queries are frequent.

3 Main Idea: Prefix Sharing

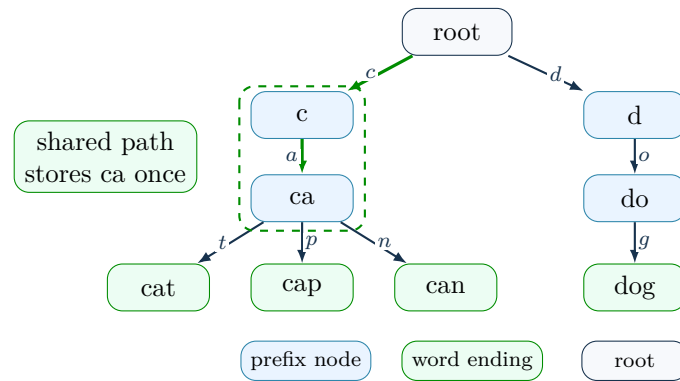
Suppose the stored words are

cat, cap, can, dog.

The words cat, cap, and can all start with the same prefix:

ca.

In a trie, that prefix is stored once, not three times.



4 Path Meaning

Every node represents the string spelled by the path from the root to that node.

Path	String represented
$\text{root} \rightarrow c$	c
$\text{root} \rightarrow c \rightarrow a$	ca
$\text{root} \rightarrow c \rightarrow a \rightarrow t$	cat
$\text{root} \rightarrow d \rightarrow o \rightarrow g$	dog

5 Complete-Word Matching

Complete-Word Search

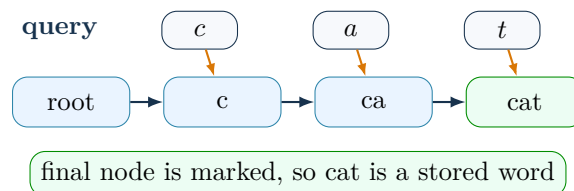
To check whether a word is stored, follow one edge per character. The word is present only if the whole path exists and the final node is marked as a word-ending node.

For the word

cat,

the search path is

$\text{root} \rightarrow c \rightarrow a \rightarrow t$.



6 Prefix Matching

Prefix Search

To check whether a prefix exists, follow one edge per prefix character. If the path exists, then at least one stored word begins with that prefix.

For prefix

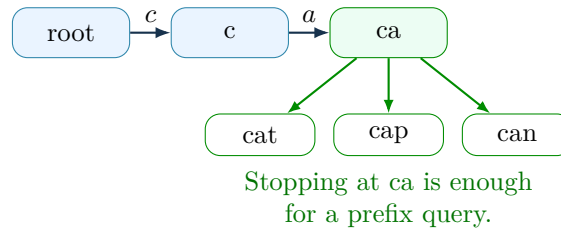
ca,

the search path is

root $\rightarrow$ c $\rightarrow$ a.

Since the path exists, the dictionary contains at least one word beginning with ca:

cat, cap, can.



Prefix Result

Path for ca exists $\Rightarrow$ there is at least one stored word with prefix ca.

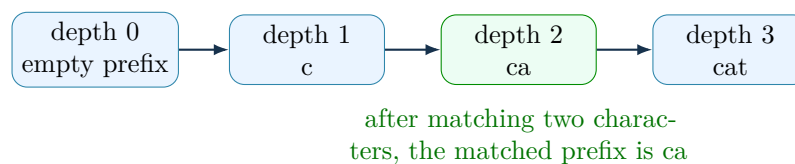
7 Depth Interpretation

Depth Rule

If a search reaches depth k , then exactly the first k characters of the query string have been matched.

depth $k \iff k$ matched characters.

For example, reaching the node for prefix ca means the search reached depth 2, so the first two characters were matched.



8 Basic Operations

Operation	Idea	Cost
Insert word w	Create missing edges while scanning characters of w .	$\mathcal{O}(w)$
Search word w	Follow existing edges for all characters and check word-ending mark.	$\mathcal{O}(w)$
Search prefix p	Follow existing edges for all prefix characters.	$\mathcal{O}(p)$

The cost depends on the length of the query string, not directly on the number of stored words.

Standard Tries

1 Definition

Standard Trie

A **standard trie** stores a set of strings using one character per edge or node transition. A root-to-terminal path spells one stored string.

A standard trie follows these rules:

- every node except the root is labelled with a character or prefix position;
- the root does not represent a normal character;
- children of a node follow a fixed canonical order;
- each stored string ends at a terminal or external node.

Path Rule

If a path from the root reads

$$c_1, c_2, \dots, c_k,$$

then the node reached by that path represents the prefix

$$c_1 c_2 \dots c_k.$$

2 Canonical Ordering

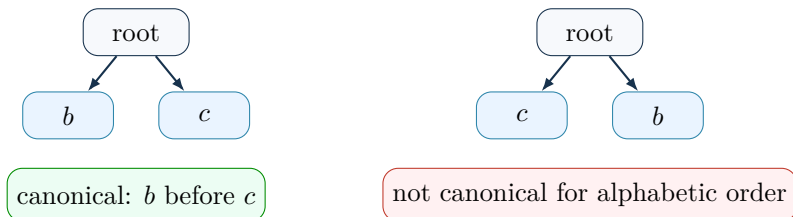
Canonical Ordering

Sibling nodes are stored in a fixed order, usually alphabetic order. This makes the trie representation consistent.

For example,

$$b < c,$$

so the child labelled b should appear before the child labelled c .



3 Example Word Set

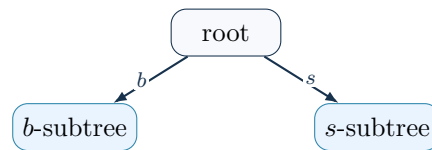
Use the set

$$S = \{\text{bear, bell, bid, bull, buy, sell, stock, stop}\}.$$

The first letter splits the words into two main groups:

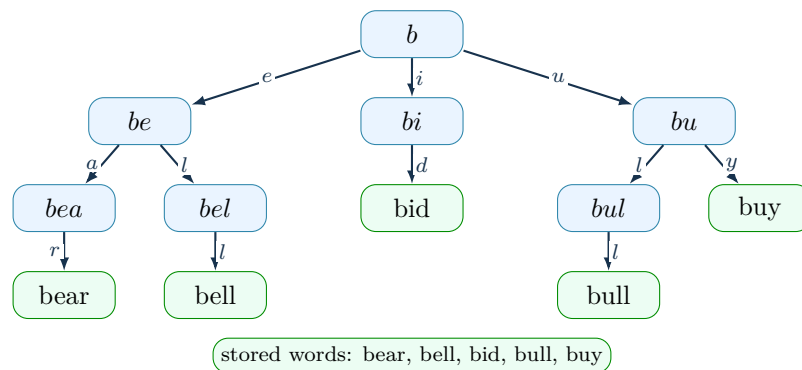
b-words and s-words.

4 Standard-Trie Diagram

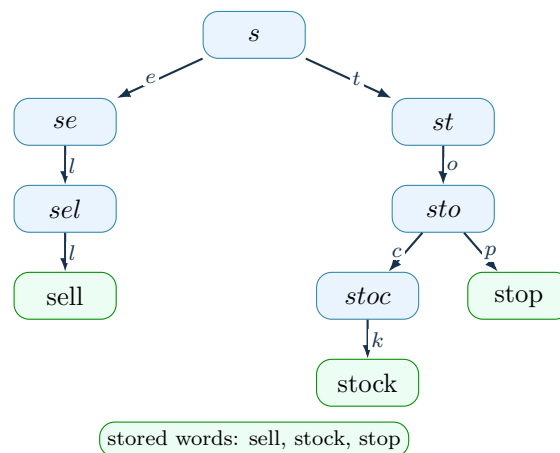


The full trie has two top-level branches. Each branch is expanded below.

b-Subtree

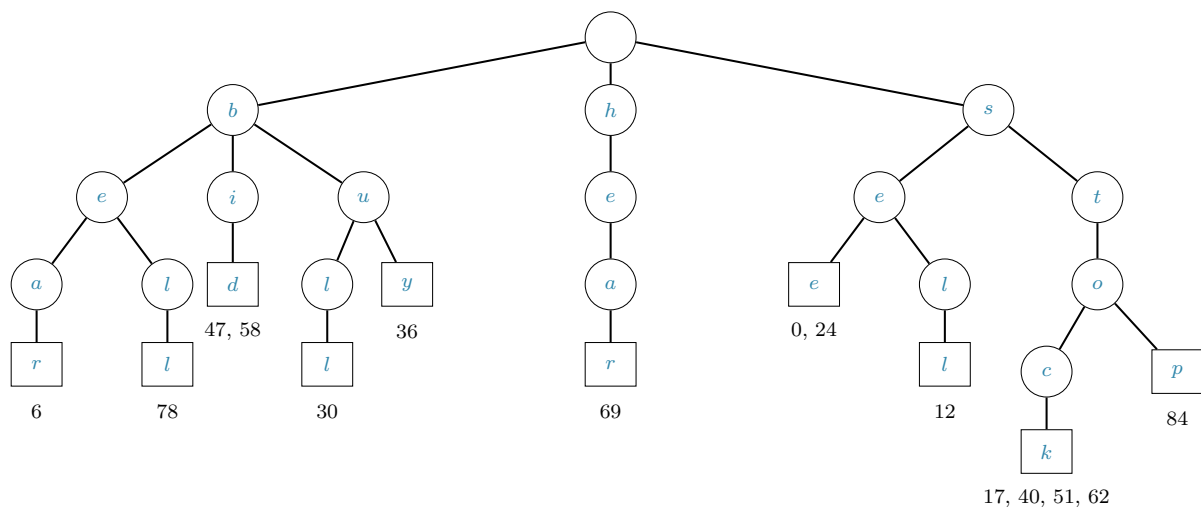
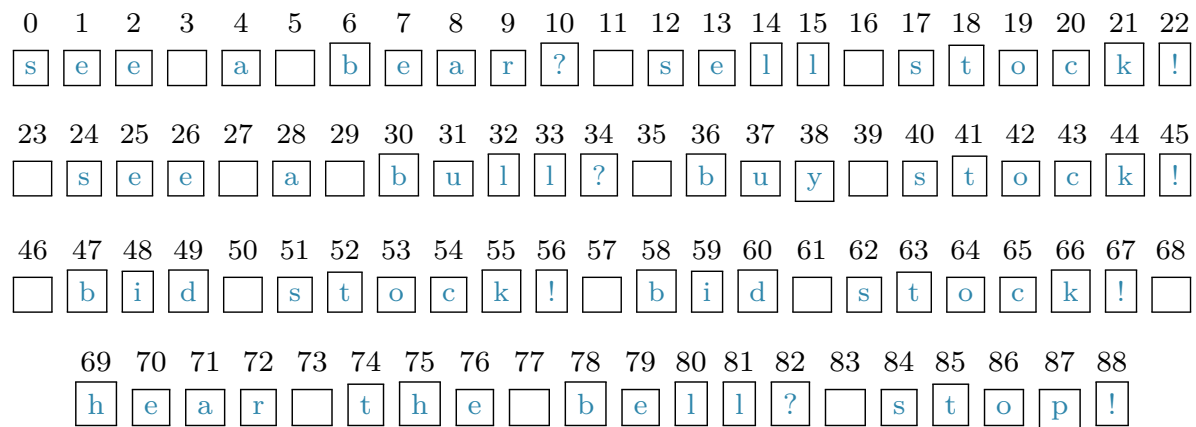


s-Subtree



5 Word-Matching Example

A trie can also store the distinct searchable words from a text. Each leaf stores the starting positions where that word appears.



Leaf Positions

The number below a leaf is the starting index of that word in the text. For example, stock appears at positions 17, 40, 51, 62.

6 Internal and External Nodes

Internal Node

An **internal node** is a non-root node that is not a leaf. It represents a partial prefix that can still be extended.

Examples from the trie above:

be, bu, sto

are internal prefix nodes.

External Node

An **external node** or leaf represents the end of a stored string.

Examples:

bear, bell, bid, bull, buy, sell, stock, stop.

If a trie stores s strings and each string has its own leaf, then

$$\text{number of external nodes} = s.$$

7 Number of Children

Let

d = size of the alphabet.

Every internal node can have between 1 and d children:

$$1 \leq \text{number of children of an internal node} \leq d.$$

For English lowercase letters,

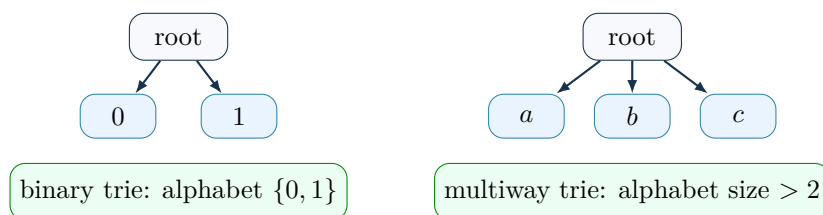
$$d = 26.$$

So a node may have up to 26 child branches.

Important Distinction

The maximum number of children depends on the alphabet size, not on the number of strings. A node in an English-letter trie cannot have more than 26 letter-labelled children.

8 Binary and Multiway Tries



Type	Alphabet	Maximum children per node
Binary trie	{0, 1}	2
Multiway trie	letters, digits, symbols, or larger alphabets	d , where d is the alphabet size

Structural Properties of Standard Tries

1 Notation

Symbols

Let S be the set of stored strings. We use:

$$\begin{aligned} s &= |S|, \\ n &= \text{total length of all strings in } S, \\ d &= \text{alphabet size}, \\ T &= \text{the standard trie built from } S. \end{aligned}$$

The total input length is

$$n = \sum_{w \in S} |w|.$$

Example

For

$$S = \{\text{cat}, \text{cap}, \text{dog}\},$$

we have

$$s = 3$$

and

$$n = |\text{cat}| + |\text{cap}| + |\text{dog}| = 3 + 3 + 3 = 9.$$

2 Maximum Degree

Maximum Degree

Every internal node in a standard trie has at most

$$d$$

children.

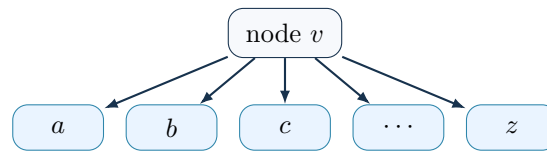
This is because each outgoing branch corresponds to one alphabet symbol. If the alphabet is

$$\Sigma = \{a, b, c, \dots, z\},$$

then

$$d = |\Sigma| = 26.$$

So an English-letter trie node can have at most 26 outgoing branches.



At most one child is needed for each possible next character.

3 External Nodes

External Node Count

If each string is represented by one terminal leaf, then a trie storing s strings has

s external nodes.

Each external node corresponds to exactly one stored string.

Example

For

$S = \{\text{bear, bell, bid, bull, buy, sell, stock, stop}\},$

there are

$$s = 8$$

stored strings, so the trie has

8 external nodes.

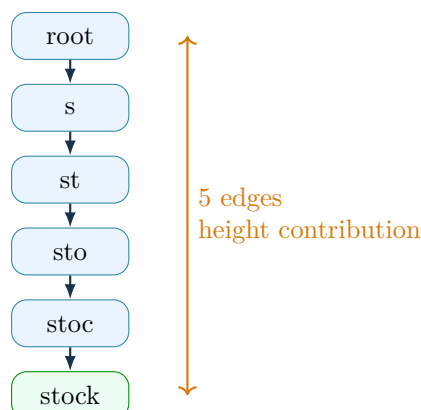
4 Height

Height of a Standard Trie

The height of a standard trie equals the length of the longest stored string:

$$h = \max_{w \in S} |w|.$$

A root-to-leaf path spells a complete stored string. Therefore, the longest root-to-leaf path corresponds to the longest word.



For the word set above, the longest word is

stock,

so

$$h = 5.$$

5 Number of Nodes

Node Bound

For a standard trie built from strings with total length n ,

$$|T| = \mathcal{O}(n).$$

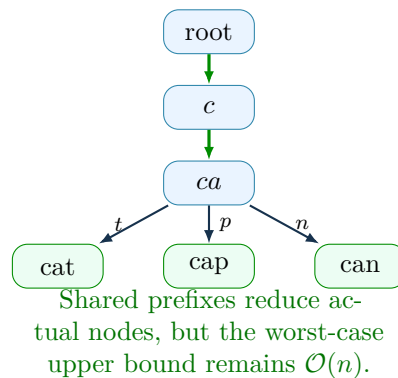
The reason is simple: while inserting all strings, each character can create at most one new trie node. Some characters create no new node because their prefix path already exists.

Prefix Sharing Example

For

cat, cap, can,

the prefix *ca* is shared.



Worst-Case Example

If stored strings share no useful prefix, little space saving occurs. For example,

cat, dog

separate immediately at the root because their first characters are different.

Even in this case, the number of nodes grows linearly with the total number of characters:

$$|\text{cat}| + |\text{dog}| = 6,$$

so the trie has only $\mathcal{O}(6)$ character-created nodes.

6 Search Complexity

Let m be the length of the word being searched. At each level, a simple implementation may check up to d child branches.

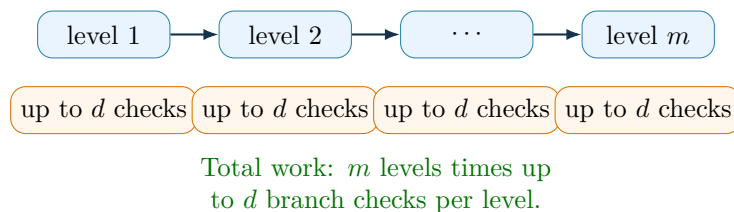
Search Bound

For a word of length m , the recorded search complexity is

$$\mathcal{O}(dm).$$

This means:

at most d child checks per level $\times m$ levels.



Implementation Note

If each node stores children in a direct array or hash table, child lookup can be faster than scanning all d children. The $\mathcal{O}(dm)$ bound corresponds to checking among up to d possible branches at each level.

Child representation	Search time for length m
Scan child list	$\mathcal{O}(dm)$
Sorted child list	$\mathcal{O}(m \log d)$
Hash table / direct array	expected $\mathcal{O}(m)$

7 Property Sheet

Property	Standard trie value
External nodes	s , one per stored string
Maximum degree	d , the alphabet size
Height	$\max_{w \in S} w $
Total nodes	$\mathcal{O}(n)$, where $n = \sum_{w \in S} w $
Search cost	$\mathcal{O}(dm)$ for query length m under branch scanning

Common Prefix Storage in Tries

1 Main Idea

Prefix Sharing

A trie stores a common prefix once. Every word that begins with that prefix reuses the same root-to-prefix path.

This is the central space-saving idea of tries. If several strings begin with the same characters, those characters are not copied into separate paths.

2 Example: cat and cats

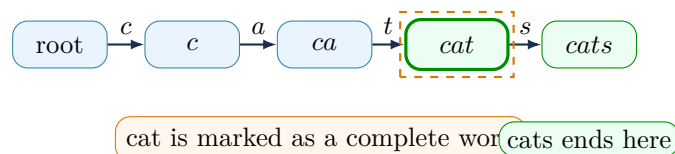
The words

cat and cats

share the prefix

cat.

The longer word continues from the same node where the shorter word ends.



Important Implementation Detail

The node for cat must be marked as a word-ending node even though it has a child for cats. A node can be both a complete word and an internal prefix node.

3 Why Sharing Saves Space

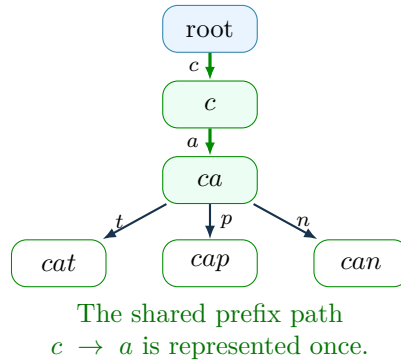
Without a trie, the strings

cat, cap, can

store the prefix ca three times:

Word	Stored prefix	Last character
cat	ca	<i>t</i>
cap	ca	<i>p</i>
can	ca	<i>n</i>

With a trie, the prefix ca is stored once:



4 Character Count Comparison

For

$$S = \{\text{cat}, \text{cap}, \text{can}\},$$

storing the words separately uses

$$|\text{cat}| + |\text{cap}| + |\text{can}| = 3 + 3 + 3 = 9$$

character occurrences.

In the trie, the distinct prefix nodes are

$$c, \quad ca, \quad cat, \quad cap, \quad can.$$

So this example uses 5 non-root prefix/word nodes instead of 9 separate character occurrences.

Savings

The saved work comes from not repeating the shared prefix *ca* for every word.

5 When Prefix Sharing Helps Most

Prefix sharing is strongest when many strings have long common prefixes.

Word set	Effect in a trie
{cat, cap, can}	Strong sharing of prefix <i>ca</i> .
{interview, internet, internal}	Strong sharing of prefix <i>inter</i> .
{cat, dog, sun}	Little sharing because first characters differ.

6 Search Benefit

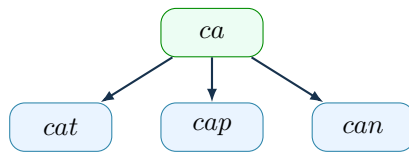
Prefix sharing also helps search. Once the prefix path is found, all words below that node share the prefix.

For example, after reaching

ca,

the subtree below that node contains exactly the words beginning with *ca*:

cat, cap, can.



A prefix query can return the whole subtree below the matched prefix.

Compressed Tries

1 Definition

Compressed Trie

A **compressed trie** removes unnecessary chains of one-child internal nodes. Instead of storing exactly one character per transition, it stores a substring on an edge.

The defining structural rule is:

Every internal node has at least two children.

This means nonbranching paths are merged into single substring-labelled edges.

2 Redundant Internal Nodes

Redundant Node

An internal node v is redundant when:

- v has exactly one child;
- v is not the root.

Such a node does not represent a branching decision. It can be removed by combining the incoming and outgoing edge labels.

3 Chain Compression

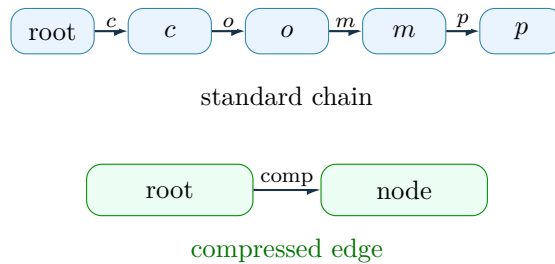
Consider a chain

$$(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k).$$

The intermediate nodes $v_1, \dots, v_{k-1}$ are redundant if each has exactly one child. The whole chain is replaced by one edge:

$$v_0 \longrightarrow v_k.$$

The new edge stores the combined substring.



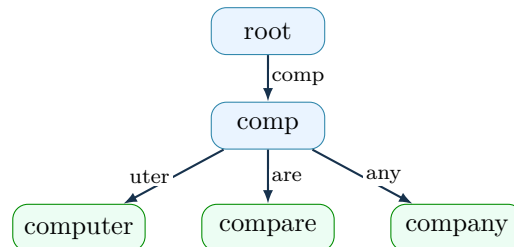
4 Common Prefix Example

For

computer, compare, company,

all three words begin with

comp.



5 Larger Example

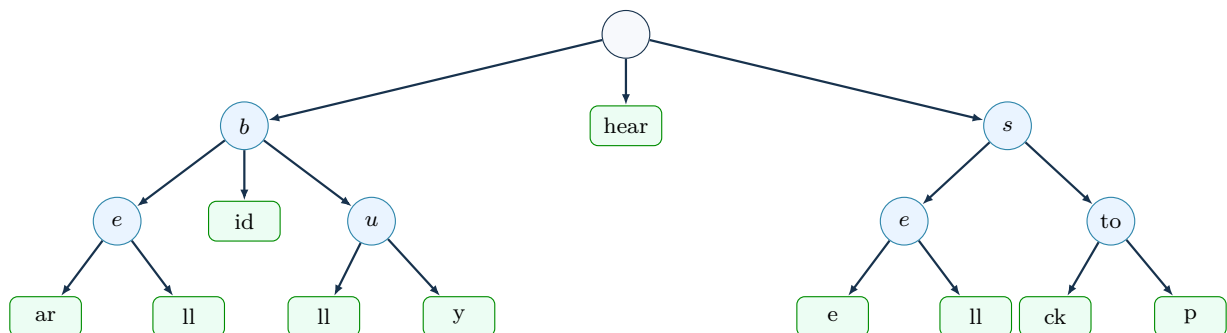
For

{see, bear, sell, stock, bull, buy, hear, bell, bid, stop},

the compressed trie can be read by top-level branches:

root branches: *b*, *h*, *s*.

Branch	Compressed continuation
<i>b</i>	$e \rightarrow ar$ for bear, $e \rightarrow ll$ for bell, id for bid, $u \rightarrow ll/y$ for bull/buy
<i>h</i>	<i>hear</i> is stored as one compressed edge
<i>s</i>	$e \rightarrow e/ll$ for see/sell, and $to \rightarrow ck/p$ for stock/stop



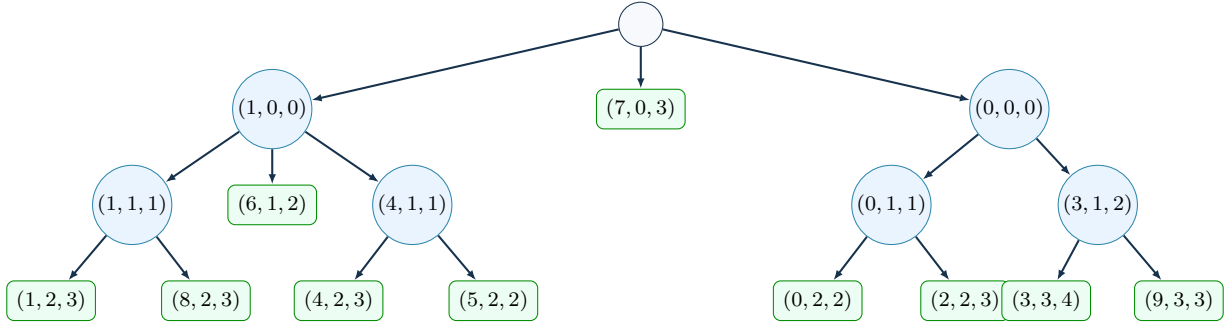
Reading the diagram

Labels such as *id*, *ar*, and *to* are compressed edge labels. They replace chains of one-character nodes.

6 Compact Index Representation

Instead of storing edge labels as strings, store references into the original string array.

Index	String	Index	String
$S[0]$	see	$S[5]$	buy
$S[1]$	bear	$S[6]$	bid
$S[2]$	sell	$S[7]$	hear
$S[3]$	stock	$S[8]$	bell
$S[4]$	bull	$S[9]$	stop



Meaning of (i, j, k)

The label (i, j, k) means the edge stores substring $S[i][j..k]$. Example: if $S[1] = \text{bear}$, then $(1, 2, 3)$ represents ar.

7 Search Behavior

When searching in a compressed trie, compare the query against whole edge labels:

edge label = substring, not one character.

Do Not Skip Edge Checks

If an edge is labelled comp, the query must match all four characters c, o, m, p before moving to the next node.

Structural Properties of Compressed Tries

1 Notation

Symbols

Let S be the set of stored strings. We use:

$$s = |S|, \quad d = |\Sigma|, \quad T = \text{compressed trie built from } S.$$

2 Degree Property

Internal-Node Degree

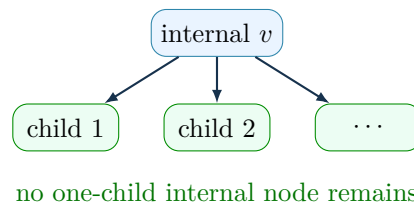
In a compressed trie, every internal node has

at least 2 children

and at most

d children.

The lower bound holds because one-child internal nodes have been compressed away. The upper bound holds because there is at most one outgoing branch per alphabet symbol.



3 External Nodes

If each stored string is represented by one leaf, then a compressed trie has

s external nodes.

Compression changes internal structure, but it does not change the fact that every stored string needs one endpoint.

4 Number of Nodes

Node Bound

For a compressed trie storing s strings,

$$|T| = \mathcal{O}(s).$$

The key reason is that every internal node branches. With s leaves and no one-child internal nodes, the number of internal branching nodes is linear in s .

5 String Labels as References

Compressed tries should not copy long substrings into every edge label. Instead, an edge label can be represented by:

- a substring reference;
- a start index;
- an end index;
- a pointer into the original text or dictionary.

Index View

A compressed trie is best treated as an index. Edges identify substrings without repeatedly storing full copies of those substrings.

Compact Edge Label

Instead of storing a full edge string, store

$$(i, j, k)$$

meaning: use substring $S[i][j..k]$ from the original string collection.

This keeps the tree small: the original strings stay in one array, and trie edges only point into that array.

6 Standard vs Compressed Trie

Feature	Standard trie	Compressed trie
Edge unit	One character	One or more characters
One-child internal nodes	Allowed	Removed
Minimum internal degree	1	2
Node count	$\mathcal{O}(n)$	$\mathcal{O}(s)$
Main benefit	Simple traversal	Reduced structural redundancy

Here

$$n = \sum_{w \in S} |w|, \quad s = |S|.$$

Suffix Trees

1 Motivation

Suffix Tree

A **suffix tree** is a compressed trie built from all suffixes of one text. It is used for fast pattern searching inside that text.

A normal trie stores a dictionary of full strings. A suffix tree stores every suffix of a single text, so any substring search can be converted into a path search.

Substring Rule

A pattern P occurs in T exactly when the characters of P can be traced from the root of the suffix tree. The leaves below the matched point give the starting positions.

2 Suffixes

For a text

$$T = t_1 t_2 \cdots t_m,$$

the suffixes are

$$T[1..m], \quad T[2..m], \quad \dots, \quad T[m..m].$$

Example

For

$$T = \text{banana},$$

some suffixes are

banana, anana, nana, ana, na, a.

3 Terminal Symbol

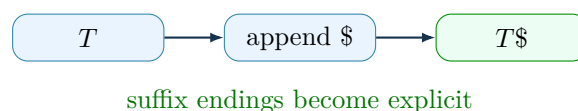
End Marker

A unique terminal symbol $\$$ is often appended to the text:

$$T\$.$$

The symbol $\$$ does not occur elsewhere in the alphabet.

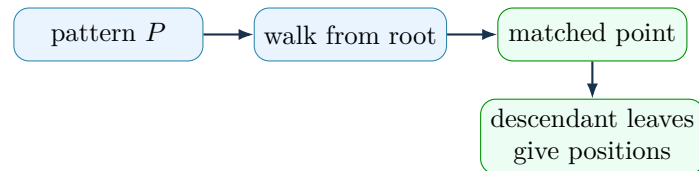
The terminal symbol ensures no suffix is accidentally treated as a prefix of another suffix.



4 Purpose

To search for a pattern P , follow the characters of P from the root across edge labels.

- If the complete pattern can be followed, then P occurs in T .
- Leaves below the matched point store starting positions of occurrences.



5 Complexity

For a text of length m , the recorded suffix-tree bounds are:

$$\boxed{\text{construction time} = \mathcal{O}(m)}$$

and

$$\boxed{\text{space} = \mathcal{O}(m)}.$$

Without compression, storing all suffixes explicitly would take

$$1 + 2 + \cdots + m = \frac{m(m+1)}{2} = \mathcal{O}(m^2)$$

characters. A compact suffix tree stores the same suffix information using only $\mathcal{O}(m)$ tree space.

Suffix-Tree Example: minimize

1 Text and Suffixes

Text

The example text is

$T = \text{minimize.}$

Its suffixes are:

Position	Suffix	With end marker
1	minimize	minimize\$
2	inimize	inimize\$
3	nimize	nimize\$
4	imize	imize\$
5	mize	mize\$
6	ize	ize\$
7	ze	ze\$
8	e	e\$

The position is important because a suffix-tree leaf can store where that suffix begins in the original text.

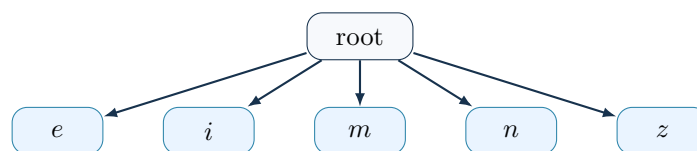
2 Root Branches

The first characters of the suffixes are

$m, i, n, i, m, i, z, e.$

Therefore, the root has branches beginning with

$e, i, m, n, z.$

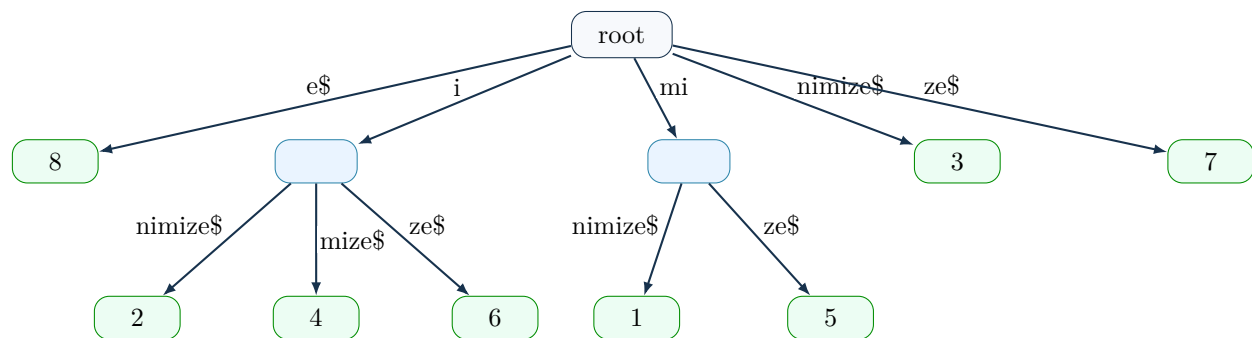


repeated first letters share one branch

3 Grouped Suffixes

Root branch	Suffixes in that group, with starting positions
<i>e</i>	8 : e\$
<i>i</i>	2 : imimize\$, 4 : imize\$, 6 : ize\$
<i>m</i>	1 : minimize\$, 5 : mize\$
<i>n</i>	3 : nimize\$
<i>z</i>	7 : ze\$

4 Full Suffix Tree



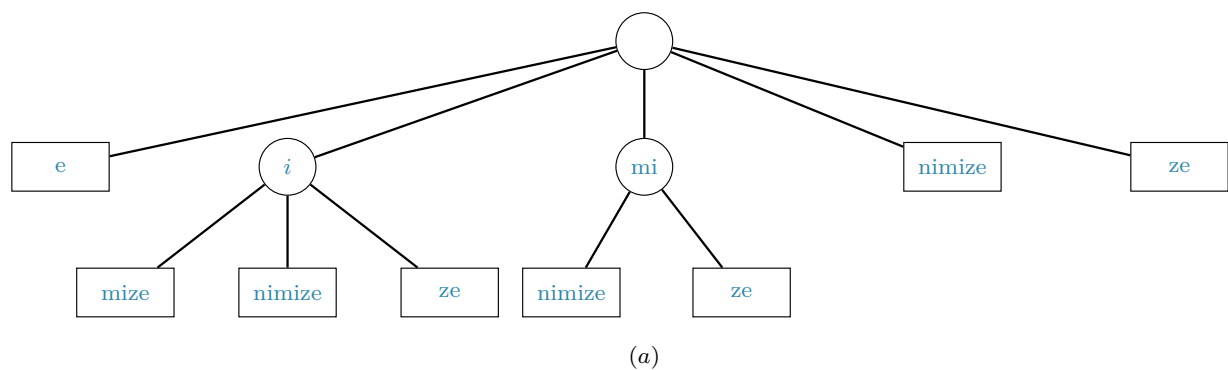
How to read the tree

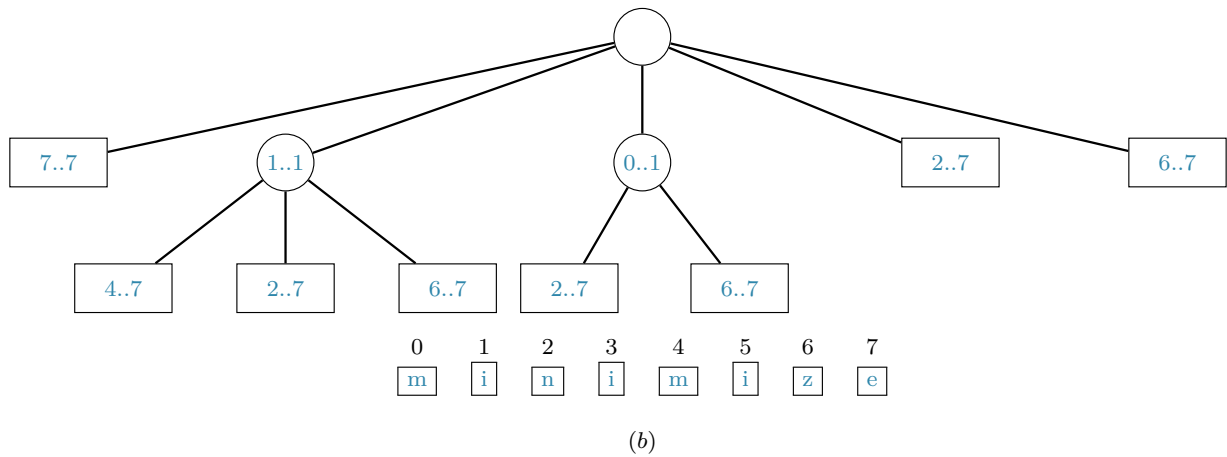
Each leaf stores the starting position of one suffix. The full suffix is obtained by reading edge labels from the root to that leaf.

5 String-Label and Index-Range Views

The same suffix tree can be drawn in two equivalent ways:

- using actual edge labels such as *nimize* and *ze*;
- using index ranges $j..k$ into the original text $T = \text{minimize}$.





Index-Range Meaning

An edge label $j..k$ means the edge stores the substring $T[j..k]$. For example, $2..7$ represents minimize, and $6..7$ represents ze.

6 Compression Idea

The suffix tree stores shared prefixes of suffixes once, and then compresses nonbranching chains into substring-labelled edges.

Interpretation

Branches such as i , m , and ze show the same compression principle used in compressed tries: common prefixes are shared, and one-child chains become longer edge labels.

7 Worked Pattern Queries

Query 1

Search for

$P = \text{mi.}$

Solution

The pattern starts with m , so search begins on the m -branch. The suffixes under that branch are

1 : minimize\$, 5 : mize\$.

Both begin with mi , so the pattern occurs at positions

1 and 5.

Query 2

Search for

$P = \text{ize.}$

Solution

The pattern starts with i , so search begins on the i -branch. Among the grouped suffixes,

6 : ize\$

starts with ize. Therefore,

ize occurs at position 6.

Suffix-Tree Example: awyawxawz

1 Text

Text

The example text is

$$T = \text{awyawxawz}.$$

The listed suffixes are:

Position	Suffix
1	awyawxawz
2	wyawxawz
3	yawxawz
4	awxawz
5	wxawz
6	xawz
7	awz
8	wz
9	z

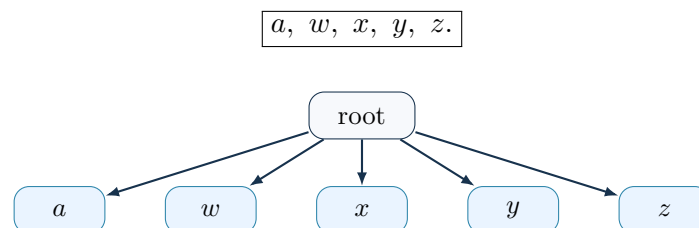
The suffix list is often written from shortest to longest, but position labels are easiest to understand in left-to-right text order.

2 Root Branches

The suffixes begin with

$a, w, y, x, z.$

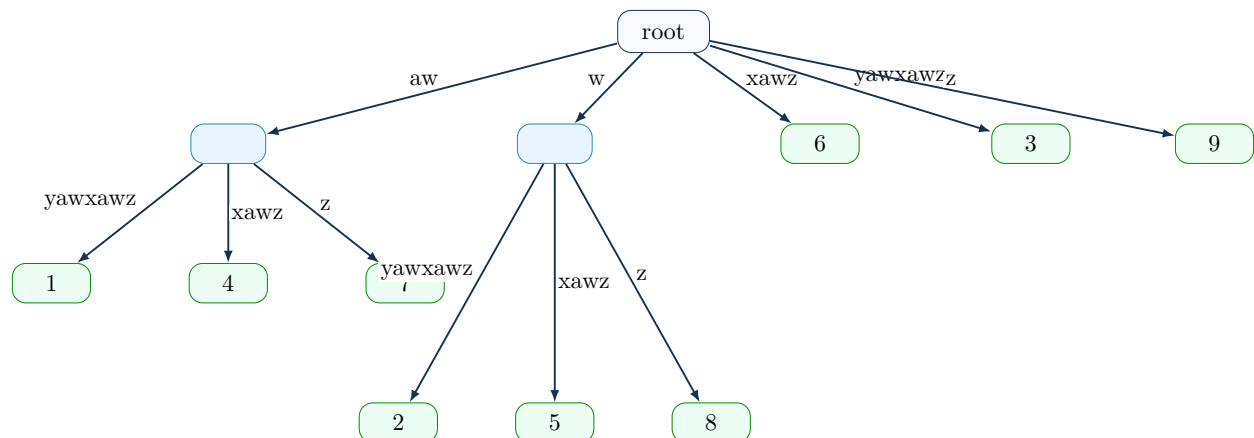
Therefore the root has branches:



3 Grouped Suffixes

Branch	Suffixes in that group, with starting positions
a	1 : awyawxawz, 4 : awxawz, 7 : awz
w	2 : wyawxawz, 5 : wxawz, 8 : wz
x	6 : xawz
y	3 : yawxawz
z	9 : z

4 Full Suffix Tree



How to read the tree

Each leaf stores a starting position. Read the edge labels from the root to a leaf to recover the suffix at that position.

5 Stored Positions

Suffix-tree leaves can store the starting positions of suffixes in the original text. For

$$T = \text{awyawxawz},$$

the suffix beginning at position 1 is awyawxawz, and the suffix beginning at position 9 is z.

Search Use

When a pattern is matched, the leaves below the matched point identify all starting positions where that pattern occurs.

6 Example Pattern

The pattern

$$P = \text{aw}$$

matches suffixes beginning with aw:

$$\text{awz}, \quad \text{awxawz}, \quad \text{awyawxawz}.$$

So the pattern occurs at the corresponding starting positions.

$$P = \text{aw} \text{ occurs at positions } 1, 4, 7.$$

7 Another Pattern

For

$$P = \text{wx},$$

search begins at the w -branch. Among the w -suffixes,

5 : wxawz

starts with wx. Hence

wx occurs at position 5.

Searching in Tries, Compressed Tries, and Suffix Trees

1 Searching in a Standard Trie

To search for a word P :

1. Start at the root.
2. Read the next character of P .
3. Follow the matching child.
4. Repeat until all characters are consumed.
5. Confirm that the final node marks the end of a stored word.

SearchTrie

```
SearchTrie(root, word):
    node = root

    for each character c in word:
        if node has no child labelled c:
            return false

        node = child labelled c

    return node marks a complete word
```

2 Prefix Search

Prefix search follows the same path, but the last node does not need to be a complete-word marker.

HasPrefix

```
HasPrefix(root, prefix):
    node = root

    for each character c in prefix:
        if node has no child labelled c:
            return false

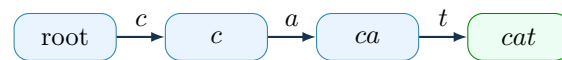
        node = child labelled c

    return true
```

Difference

Word search asks: “Does this exact word exist?” Prefix search asks: “Does any stored word begin with this prefix?”

3 Standard-Trie Search Trace



query cat succeeds if final node is terminal

4 Searching in a Compressed Trie

In a compressed trie, an edge may contain a substring. The search must compare the query against the entire edge label.

SearchCompressedTrie

```
SearchCompressedTrie(root, word):
    node = root
    position = 0

    while position < length(word):
        choose an outgoing edge whose first character
        matches word[position]

        if no such edge exists:
            return false

        compare the edge label with the corresponding
        substring of word

        if the full edge label does not match:
            return false

        advance position by length(edge label)
        node = edge destination

    return final position represents a stored word
```



compare whole edge labels, not only one character

5 Searching in a Suffix Tree

To search for a pattern P :

1. Start at the root.
2. Choose an edge whose first character matches the next character of P .
3. Compare P against the edge label.
4. Continue until all of P is matched or a mismatch occurs.

If P is fully matched, then the descendant leaves identify where P occurs in the original text.

6 Comparison

Structure	Search rule
Standard trie	Compare one character per edge.
Prefix search	Same as word search, but no terminal mark required at the final node.
Compressed trie	Compare an entire substring edge label before moving on.
Suffix tree	Match a pattern across edge labels; descendant leaves give occurrences.

7 Search-Engine Indexing Idea

Inverted Index

For each keyword, store the list of document positions where it appears. A compressed trie can store the keywords, and each leaf points to that keyword's occurrence list.

Query type	How it is answered
One keyword	Search the keyword in the trie, then return its occurrence list.
Several keywords	Retrieve all occurrence lists and intersect them.
Prefix query	Walk to the prefix node and collect descendant keyword leaves.

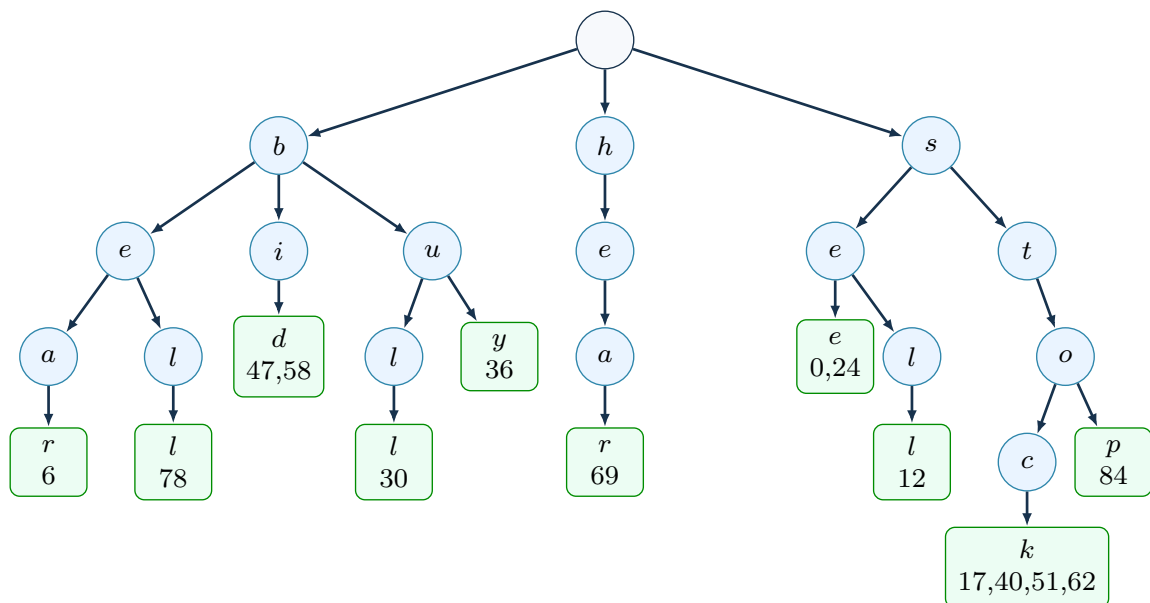


Diagram Meaning

The path spells the word; the leaf stores where that word begins. For example, the path stock ends with the occurrence list 17, 40, 51, 62.

Why store lists outside the trie?

Occurrence lists can be very large, so the trie stores only the dictionary structure and pointers to lists. The lists themselves are stored separately.